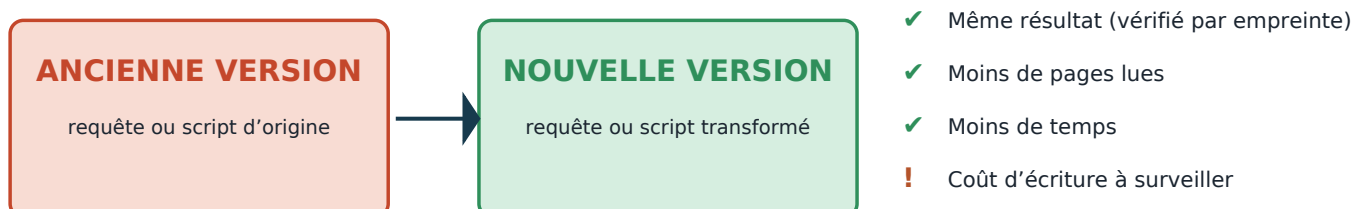


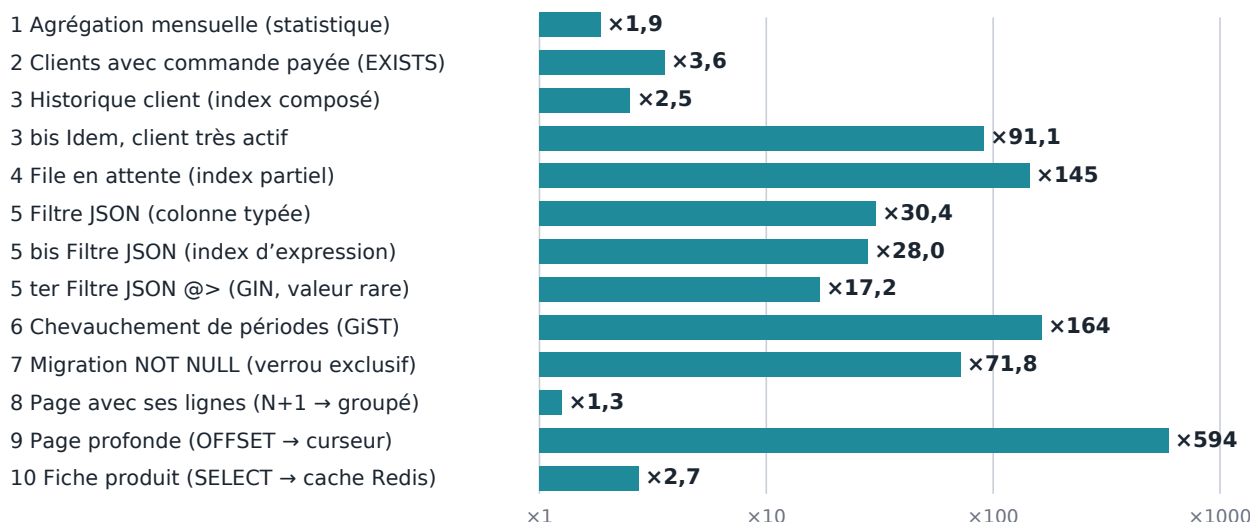
Optimiser une requête : le même résultat, pour moins cher

Mes optimisations sur ShopFlow (PostgreSQL 18) : ancienne requête → nouvelle requête

Ce que j'appelle optimiser. Je pars d'une **ancienne requête** (ou d'un ancien script) et je la transforme en une **nouvelle version** qui donne **exactement le même résultat** mais s'exécute **plus vite et pour moins cher**. Chaque page montre cette transformation, la preuve que le résultat est identique, le schéma du plan d'exécution et le gain mesuré.



Gain de vitesse, résultat identique (échelle logarithmique)



#	Optimisation	Ancienne → nouvelle version	Résultat identique
1	Agrégation mensuelle (statistique)	84,6 → 45,5 ms (x1,9)	✓ md5
2	Clients avec commande payée (EXISTS)	40,2 → 11,3 ms (x3,6)	✓ 0 écart
3	Historique client (index composé)	0,690 → 0,274 ms (x2,5)	✓ md5 (75 cas)
3 bis	Idem, client très actif	23,3 → 0,256 ms (x91,1)	✓ md5
4	File en attente (index partiel)	10,1 → 0,070 ms (x145)	✓ md5
5	Filtre JSON (colonne typée)	30,9 → 1,0 ms (x30,4)	✓ md5
5 bis	Filtre JSON (index d'expression)	30,9 → 1,1 ms (x28,0)	✓ md5
5 ter	Filtre JSON @> (GIN, valeur rare)	43,9 → 2,6 ms (x17,2)	✓ md5
6	Chevauchement de périodes (GiST)	27,5 → 0,168 ms (x164)	✓ md5
7	Migration NOT NULL (verrou exclusif)	3 324 → 46,3 ms (x71,8) *	✓ md5
8	Page avec ses lignes (N+1 → groupé)	55,3 → 43,9 ms (x1,3)	✓ réponse JSON
9	Page profonde (OFFSET → curseur)	99,7 → 0,168 ms (x594) *	✓ md5 (8 cas)
10	Fiche produit (SELECT → cache Redis)	2,7 → 0,988 ms (x2,7) *	✓ data identique

* 7 : temps où la table est inaccessible (3 000 000 lignes, une exécution). 9 : 1 000 000 de commandes, décalage 100 000. 10 : 100 paires, copie qui peut être périmée. Médianes variables d'une exécution à l'autre. Mesures brutes : Atelier1_Diagnostic_compact.pdf.

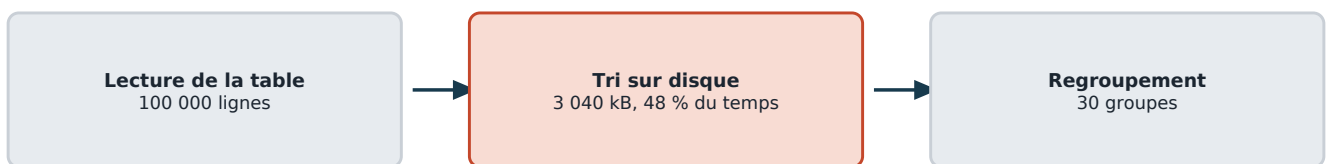
1. Agrégation mensuelle : corriger l'estimation de PostgreSQL

Même requête, même résultat, ×1,9 plus rapide

Le problème. L'indicateur mensuel regroupe 100 000 commandes en 30 groupes. PostgreSQL **croit qu'il y aura 80 048 groupes** (il ne sait pas compter les mois), choisit donc de trier les 100 000 lignes, et ce tri **déborde sur le disque** (3 040 kB).

ANCIENNE VERSION	NOUVELLE VERSION
<pre>SELECT date_trunc('month', created_at) AS mois, statut, count(*), sum(total) FROM commandes GROUP BY mois, statut ORDER BY mois, statut; -- Estimation : 80 048 groupes (réel : 30)</pre>	<pre>-- La requête ne change pas. On ajoute : CREATE STATISTICS st_mois ON (date_trunc('month', created_at)), statut FROM commandes; ANALYZE commandes; -- Estimation : 30 groupes</pre>

ANCIENNE VERSION : ce que fait PostgreSQL



NOUVELLE VERSION : ce que fait PostgreSQL



Durée de la requête (ms)

Ancienne  84,6 ms

Statistique  45,5 ms (×1,9)

Plus de mémoire  50,2 ms (×1,7)

Où va le temps (avant)

Tri sur disque  48 %

Lecture table  33 %

Regroupement  20 %

Résultat identique ✓ Empreinte md5 du résultat complet (30 lignes) identique pour la référence, la statistique et le work_mem : 601f7d5aefe1.

Pourquoi ça marche	Ce que ça coûte, et les limites
Avec la statistique, PostgreSQL sait qu'il n'y a que 30 groupes : il les compte dans une petite table en mémoire (40 kB) au lieu de trier 100 000 lignes sur disque. On corrige la cause (l'estimation).	La statistique est peu coûteuse (une donnée à maintenir à chaque ANALYZE ; coût non mesuré). Donner plus de mémoire (work_mem 32 MB) marche aussi (×1,7) mais masque le symptôme et réserve plus de mémoire à chaque opération. Mesures faites sur une copie de la base.

À retenir. Une requête peut être lente parce que PostgreSQL se trompe, pas parce qu'elle est mal écrite. On l'accélère sans changer son SQL.

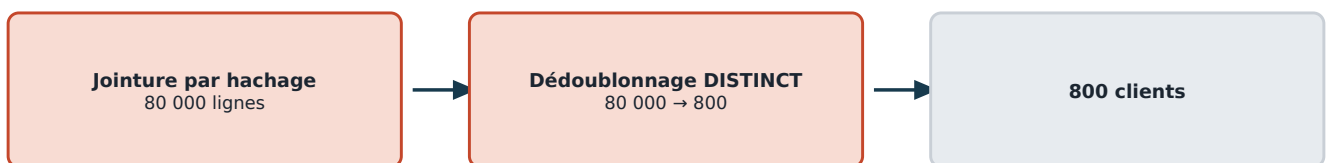
2. Clients ayant une commande payée : réécrire la requête

Même résultat (800 clients), ×3,6 plus rapide

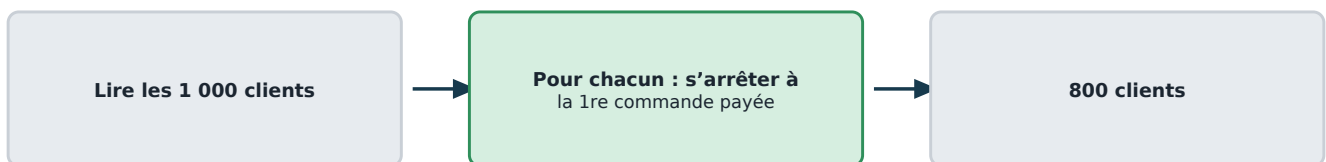
Le problème. L'ancienne requête joint les clients à leurs commandes : elle construit **une ligne par commande** (80 000 lignes pour 800 clients, le client 42 apparaît 100 fois), puis DISTINCT supprime les doublons après coup.

ANCIENNE VERSION	NOUVELLE VERSION
<pre>SELECT DISTINCT cl.id, cl.nom FROM clients cl JOIN commandes c ON c.client_id = cl.id WHERE c.statut = 'payee';</pre>	<pre>SELECT cl.id, cl.nom FROM clients cl WHERE EXISTS (SELECT 1 FROM commandes c WHERE c.client_id = cl.id AND c.statut = 'payee');</pre>

ANCIENNE VERSION : ce que fait PostgreSQL



NOUVELLE VERSION : ce que fait PostgreSQL



Durée (ms)

Ancienne  40,2 ms

Nouvelle  11,3 ms (×3,6)

Lignes construites

Ancienne  80 000

Nouvelle  800

Résultat identique ✓ La différence symétrique entre l'ancienne et la nouvelle requête (EXCEPT dans les deux sens) renvoie **0 ligne** : ce sont exactement les mêmes 800 clients.

Pourquoi ça marche	Ce que ça coûte, et les limites
La nouvelle requête pose une question de présence (« y a-t-il au moins une commande payée ? ») : PostgreSQL la traite en semi-jointure et arrête la recherche dès la première commande trouvée, sans jamais fabriquer les 80 000 lignes.	Aucun coût : pas d'index, pas d'écriture en plus. Condition : on veut un résultat par client , pas une ligne par commande. Une transformation qui retirerait seulement DISTINCT changerait le résultat : elle est écartée (voir la page des transformations écartées).

À retenir. Réécrire la requête sans changer son résultat : on évite de construire 80 000 lignes pour n'en garder que 800.

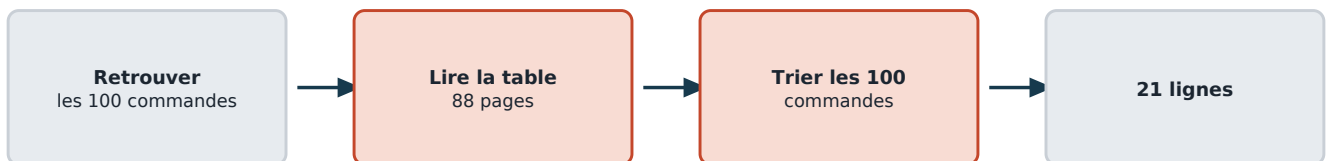
3. Historique d'un client : l'index composé

Même résultat, ×2,5 plus rapide, et ×91,1 pour un client très actif

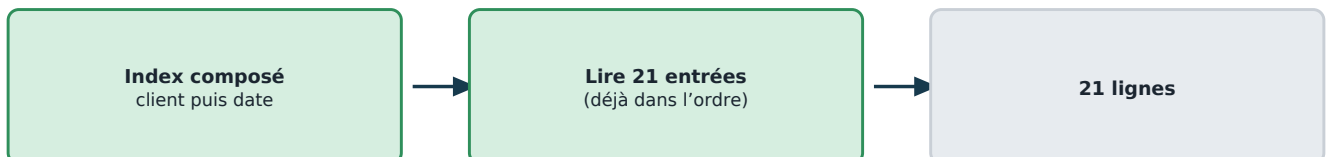
Le problème. L'API affiche les 21 dernières commandes d'un client. PostgreSQL retrouve les 100 commandes du client, les lit dans la table (dispersées), puis les **trie** pour n'en garder que 21. Plus le client a de commandes, plus c'est long.

ANCIENNE VERSION	NOUVELLE VERSION
<pre>SELECT id, created_at, statut, total FROM commandes WHERE client_id = 42 ORDER BY created_at DESC, id DESC LIMIT 21;</pre>	<pre>-- La requête ne change pas. On crée : CREATE INDEX idx_hist_client ON commandes (client_id, created_at DESC, id DESC);</pre>

ANCIENNE VERSION : ce que fait PostgreSQL



NOUVELLE VERSION : ce que fait PostgreSQL



Durée (ms), client à 100 commandes

Ancienne  0,690 ms

Nouvelle  0,274 ms (×2,5)

Durée (ms), client à 20 100 commandes

Ancienne  23,3 ms

Nouvelle  0,256 ms (×91,1)

Résultat identique ✓ Empreinte md5 du résultat identique pour **75 combinaisons** (5 variantes × 3 requêtes × 5 clients), par exemple client 42 : 21df3003a95c. Pages lues : 96 → 27.

Pourquoi ça marche	Ce que ça coûte, et les limites
L'égalité sur <code>client_id</code> isole les commandes du client, et l'index les range déjà par date : PostgreSQL lit 21 entrées et s'arrête, sans trier. Sans index, le coût grandit avec le client (287 pages à 20 100 commandes) ; avec l'index il reste constant (27).	Index de 3 984 kB (30 % de la table) et WAL +35,0 % à l'insertion. Un index simple sur <code>client_id</code> n'apporte rien (même plan) ; si l' <code>ORDER BY</code> n'est pas préfixé du nom de la table, le tri porte sur du texte et l'index devient inutile .

À retenir. L'index ne change pas la requête : il supprime le travail de tri. Le gain croît avec le volume.

4. File des commandes à traiter : l'index partiel

Même résultat, ×145 plus rapide, avec un index 12 fois plus petit

Le problème. Un opérateur traite les 100 plus anciennes commandes en_attente, soit 10 % de la table. PostgreSQL lit les **100 000 commandes**, ne retient que 10 000, puis les trie.

ANCIENNE VERSION	NOUVELLE VERSION
<pre>SELECT id FROM commandes WHERE statut = 'en_attente' ORDER BY created_at, id LIMIT 100;</pre>	<pre>-- La requête ne change pas. On crée : CREATE INDEX idx_attente ON commandes (created_at, id) WHERE statut = 'en_attente';</pre>

ANCIENNE VERSION : ce que fait PostgreSQL



NOUVELLE VERSION : ce que fait PostgreSQL



Durée (ms)

Ancienne  10,1 ms

Index partiel  0,070 ms (×145)

Taille de l'index (kB)

Index complet  4 072 kB

Index partiel  328 kB

Résultat identique ✓ Empreinte md5 du résultat (100 lignes) identique à la base initiale : 7a826168f2fd. Pages lues : 1 680 → 3.

Pourquoi ça marche	Ce que ça coûte, et les limites
L'index ne contient que les 10 000 commandes en attente, rangées par date : PostgreSQL lit les 100 premières et s'arrête. Il est 12,4 fois plus petit qu'un index sur tous les statuts.	Écriture : WAL +3,2 % à l'insertion (contre +36,5 % pour un index complet). Limites : il ne sert que en_attente (pas payee ni annulee), et seulement si la requête contient le littéral : avec un paramètre (plan générique) il n'est pas utilisé.

À retenir. Un index peut être petit et très efficace s'il ne couvre que le besoin, à condition que la requête le lui permette.

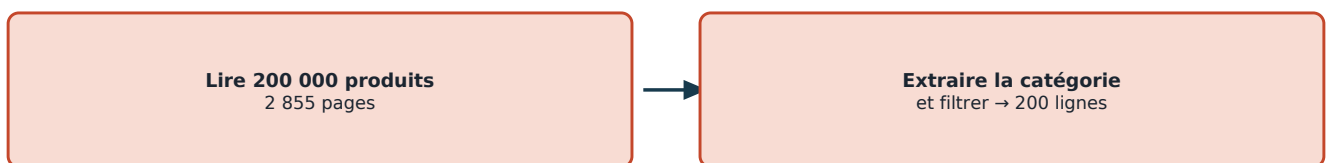
5. Filtre sur les attributs JSON : sortir la catégorie du JSON

Même résultat, ×30,4 plus rapide sur 200 000 produits

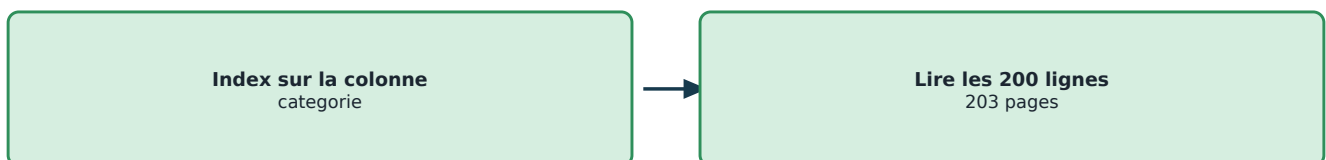
Le problème. Les attributs variables sont stockés en JSONB. Filtrer sur la catégorie oblige PostgreSQL à **lire les 200 000 produits** et à extraire la catégorie de chaque ligne. (Sur les 200 produits réels, un Seq Scan sur 3 pages est le bon choix : aucun index n'y est utilisé.)

ANCIENNE VERSION	NOUVELLE VERSION
<pre>SELECT id, nom FROM produits WHERE attributs->>'categorie' = 'rare'; -- Seq Scan sur 200 000 lignes -- (mesuré avec count(*) et sum(id))</pre>	<pre>ALTER TABLE produits ADD COLUMN categorie text GENERATED ALWAYS AS (attributs->>'categorie') STORED; CREATE INDEX ON produits (categorie); SELECT id, nom FROM produits WHERE categorie = 'rare';</pre>

ANCIENNE VERSION : ce que fait PostgreSQL



NOUVELLE VERSION : ce que fait PostgreSQL



Durée (ms)

Ancienne 30,9 ms

Nouvelle 1,0 ms (×30,4)

Pages lues

Ancienne 2 855

Nouvelle 203

Résultat identique ✓ count(*) et sum(id) identiques entre l'ancienne et la nouvelle requête (empreinte md5 11157d484cf8) : ce sont les mêmes 200 produits.

Pourquoi ça marche	Ce que ça coûte, et les limites
La catégorie devient une vraie colonne indexée : PostgreSQL va directement aux 200 lignes concernées. Pour une propriété structurante et toujours filtrée, c'est la transformation que recommande le cours (une colonne typée plutôt que du JSON).	Changement de schéma (ALTER TABLE), table +1,80 MiB, index de 1,37 MiB, WAL +38,9 % à l'insertion. Limite : le filtre doit porter toujours sur cette propriété. Même résultat aussi sans toucher à la requête : un index d'expression (×28,0, WAL +34,6 %) ; pour les requêtes avec l'opérateur @>, un index GIN (×17,2 sur une valeur rare, mais WAL +120,7 %).

À retenir. Quand une propriété est toujours filtrée, la sortir du JSON dans une colonne indexée transforme une lecture complète en accès direct.

6. Détection de chevauchement de périodes

Même résultat, ×164 plus rapide sur 200 000 périodes

Le problème. Pour savoir quelles périodes chevauchent une fenêtre, PostgreSQL teste les **200 000 périodes une à une**. (Ce besoin n'existe pas encore dans ShopFlow : l'exemple est indépendant de l'application.)

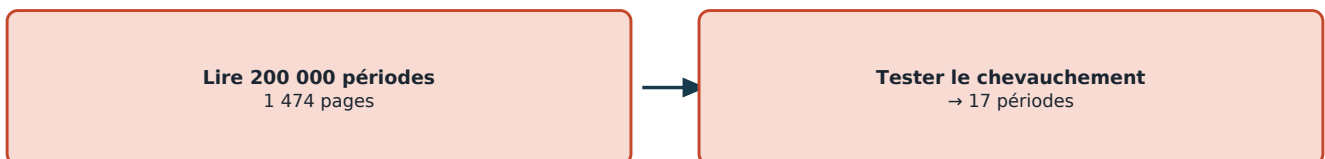
ANCIENNE VERSION

```
SELECT id FROM reservations
WHERE periode &&
      tstzrange('2026-03-01 14:00+00',
                '2026-03-01 16:00+00');
-- Seq Scan sur 200 000 lignes
```

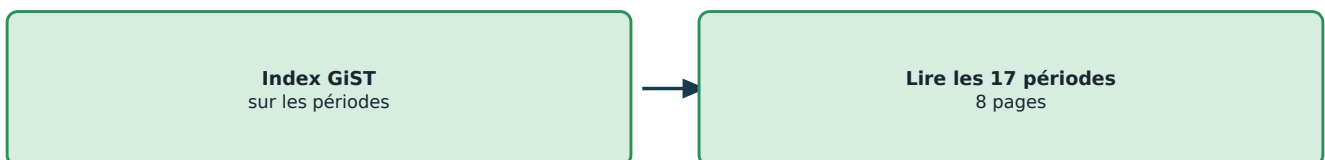
NOUVELLE VERSION

```
-- La requête ne change pas. On crée :
CREATE INDEX idx_periode
ON reservations
USING gist (periode);
```

ANCIENNE VERSION : ce que fait PostgreSQL



NOUVELLE VERSION : ce que fait PostgreSQL



Durée (ms)

Ancienne  27,5 ms

Nouvelle  0,168 ms (×164)

Pages lues

Ancienne  1 474

Nouvelle  8

Résultat identique ✓ Empreinte md5 identique (e6f0c0d64f01) : mêmes 17 périodes. Sur le petit jeu de test, la période qui touche la fenêtre sans la recouvrir (16h-17h contre 14h-16h) n'est pas retournée : les intervalles sont semi-ouverts.

Pourquoi ça marche

L'index GiST range les périodes de façon à retrouver celles qui se chevauchent sans toutes les tester : il prend en charge l'opérateur de chevauchement &&.

Ce que ça coûte, et les limites

WAL +54,1 % à l'insertion, insertions ×7,8 plus lentes, index de 6,95 MiB (60,5 % de la table). À réserver aux cas où les recherches de conflit sont bien plus fréquentes que les insertions.

À retenir. Un index se choisit d'après l'opérateur de la requête : ici && demande un GiST.

7. Ajouter une colonne NOT NULL à une grande table : migrer sans bloquer

Même état final, table inaccessible **x71,8 moins longtemps (3 324 → 46,3 ms)**

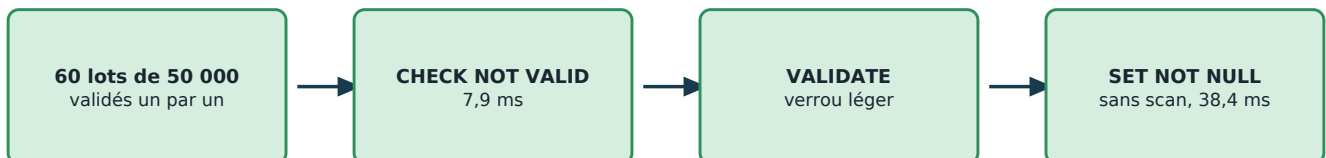
Le problème. Ajouter newsletter_ok (NOT NULL, défaut false) à 3 000 000 lignes pendant que l'application tourne. L'approche directe fait un seul UPDATE (écrivains en attente jusqu'à 30 989 ms), puis SET NOT NULL **relit toute la table sous verrou exclusif : aucune lecture ne passe pendant 3 324 ms.**

ANCIENNE VERSION	NOUVELLE VERSION
<pre>ALTER TABLE clients ADD COLUMN newsletter_ok boolean; ALTER TABLE clients ALTER COLUMN newsletter_ok SET DEFAULT false; UPDATE clients SET newsletter_ok = false; -- un seul UPDATE, tout le volume ALTER TABLE clients ALTER COLUMN newsletter_ok SET NOT NULL; -- relit toute la table</pre>	<pre>ALTER TABLE clients ADD COLUMN newsletter_ok boolean; ALTER TABLE clients ALTER COLUMN newsletter_ok SET DEFAULT false; -- lots validés un par un, jusqu'à 0 : WITH lot AS (SELECT id FROM clients WHERE newsletter_ok IS NULL ORDER BY id LIMIT n) UPDATE clients c SET newsletter_ok = false FROM lot WHERE c.id = lot.id AND c.newsletter_ok IS NULL; ALTER TABLE clients ADD CONSTRAINT nn CHECK (newsletter_ok IS NOT NULL) NOT VALID; ALTER TABLE clients VALIDATE CONSTRAINT nn; ALTER TABLE clients ALTER COLUMN newsletter_ok SET NOT NULL; -- saute le scan</pre>

ANCIENNE VERSION : ce que fait PostgreSQL



NOUVELLE VERSION : ce que fait PostgreSQL



Temps sous verrou exclusif (ms)

Ancienne  3 324 ms

Nouvelle  46,3 ms (x71,8)

Écriture la plus longue bloquée (ms)

Ancienne  30 989 ms

Nouvelle  416 ms (x74,5)

Résultat identique ✓ Empreinte md5 du contenu complet identique (b227f2377b62) : 3 000 000 lignes, toutes à false, 0 NULL, colonne NOT NULL, défaut false, dans les deux cas. Sur la copie de 1 000 clients de l'atelier : 0 écart (md5 aec47d5bdba0).

Pourquoi ça marche	Ce que ça coûte, et les limites
CHECK ... NOT VALID ne relit rien ; VALIDATE relit la table avec un verrou léger (lectures, UPDATE et INSERT passent : vérifié). Le CHECK validé prouve l'absence de NULL : SET NOT NULL saute le scan (message DEBUG1 relevé). Le verrou exclusif ne dure que quelques ms.	La nouvelle méthode n'est pas plus rapide au total : remplissage 57 s → 156 s (≈ x2,7), WAL x1,22, plus d'étapes à gérer. On échange du temps total contre l'absence de blocage. Une seule exécution (durées variables) ; lots de 50 000 lignes ici, 200 dans l'atelier.

À retenir. Optimiser une migration, c'est réduire le temps pendant lequel les autres utilisateurs attendent, pas forcément la durée totale.

8. Page de commandes et ses lignes : sortir du N+1

Même page, même réponse : 21 requêtes SQL → 2 (55,3 → 43,9 ms)

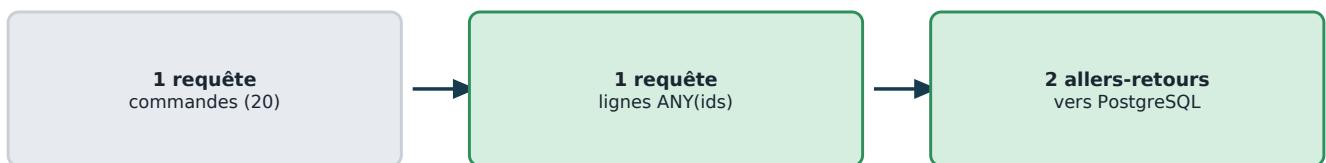
Le problème. L'écran du client 42 affiche 20 commandes avec leurs lignes. Pour chaque commande, l'API envoie **une requête de lignes** : $1 + 20 = 21$ **requêtes**. Le nombre de requêtes **croît avec la taille de la page** (6, 21, 51, 101 pour 5, 20, 50, 100 commandes).

ANCIENNE VERSION	NOUVELLE VERSION
<pre>rows = SELECT ... FROM commandes WHERE client_id = 42 LIMIT 20; POUR CHAQUE commande de rows : SELECT ... FROM lignes WHERE commande_id = \$1; -- 1 + 20 = 21 requêtes</pre>	<pre>rows = SELECT ... FROM commandes WHERE client_id = 42 LIMIT 20; SELECT ... FROM lignes WHERE commande_id = ANY(\$1::bigint[]); -- \$1 = les 20 ids de la page -- regrouper en mémoire -- 2 requêtes, quelle que soit la page</pre>

ANCIENNE VERSION : ce que fait PostgreSQL



NOUVELLE VERSION : ce que fait PostgreSQL



Requêtes SQL (page de 20)

Ancienne 21

Nouvelle 2 (×10,5)

Durée HTTP médiane (ms, 30 tours)

Ancienne 55,3 ms

Nouvelle 43,9 ms (×1,3)

Résultat identique ✓ La réponse JSON complète (commandes **et** lignes, champ par champ) est identique en mode N+1 et en mode groupé pour 5, 20, 50 et 100 commandes ; chaque commande a bien ses 3 lignes. Une page vide ne déclenche qu'une requête dans les deux modes.

Pourquoi ça marche	Ce que ça coûte, et les limites
Les lignes de toute la page sont lues en une seule requête sur les identifiants, puis regroupées par commande en mémoire : le nombre de requêtes ne dépend plus de la taille de la page. Page de 50 : 51 → 2 requêtes, durée HTTP médiane 68,5 → 44,2 ms (×1,5).	Le gain de durée est modeste sur cette base locale (11,4 ms pour 20 commandes) : chaque aller-retour coûte peu ; il devrait grandir avec la latence du réseau (non mesuré). Le tableau d'ids doit rester borné par la taille maximale de page (100). Une jointure unique aurait été une autre option (non testée : elle multiplie les lignes et interagit avec LIMIT).

À retenir. Avant de parler de durée, on compte les requêtes : N+1 se voit au nombre de requêtes qui croît avec N, et se corrige sans changer la réponse.

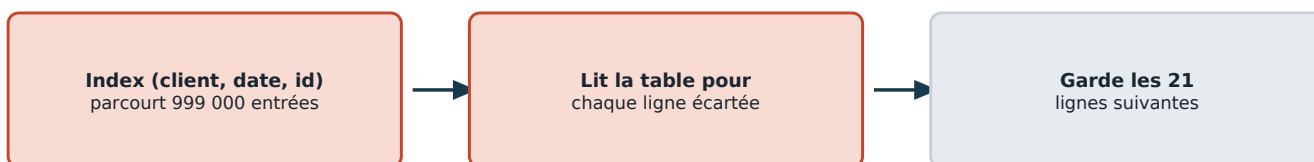
9. Pagination profonde : du OFFSET au curseur

Même page de 20 : 944,3 ms → 0,177 ms (×5335) à 1 000 000 de commandes

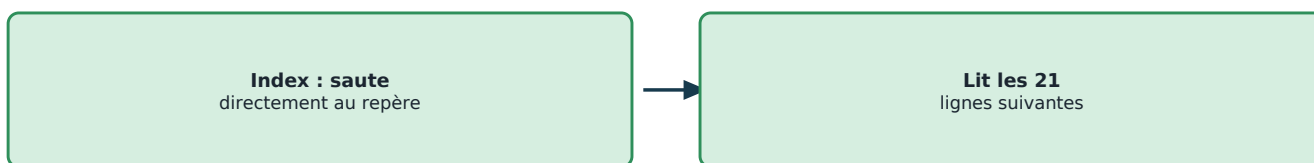
Le problème. Pour afficher la page 49 951 (décalage 999 000), OFFSET demande à PostgreSQL de **lire puis écarter** toutes les lignes d'avant : le travail croît avec le décalage. (Mesuré sur une table de travail : le client 42 n'a que 100 commandes, ce qui ne permet pas de voir l'effet.)

ANCIENNE VERSION	NOUVELLE VERSION
<pre>SELECT id, created_at, statut, total FROM commandes WHERE client_id = 42 ORDER BY created_at DESC, id DESC LIMIT 21 OFFSET 999000; -- lit ≈ 1 000 000 lignes, en garde 21</pre>	<pre>SELECT id, created_at, statut, total FROM commandes WHERE client_id = 42 AND (created_at, id) < (\$2, \$3) ORDER BY created_at DESC, id DESC LIMIT 21; -- (\$2, \$3) = repère de la dernière -- ligne affichée (le curseur)</pre>

ANCIENNE VERSION : ce que fait PostgreSQL



NOUVELLE VERSION : ce que fait PostgreSQL



Durée (ms), décalage 999 000

Ancienne **944,3 ms**

Nouvelle **0,177 ms**

Pages lues, décalage 999 000

Ancienne **1 003 945**

Nouvelle **24**

Résultat identique ✓ Les deux requêtes renvoient la **même page** (empreinte md5 identique) à chacune des 8 profondeurs testées (0 à 999 000), sur des dates en double (deux commandes par date). Sur l'API, la page 2 est identique par OFFSET et par curseur, et le parcours des 5 pages donne 100 commandes uniques dans l'ordre de PostgreSQL.

Pourquoi ça marche	Ce que ça coûte, et les limites
Le curseur est la paire (date, id) de la dernière ligne vue : l'index reprend à ce repère et lit seulement la page, quel que soit le décalage. L'id départage les dates identiques (sans lui, la commande 42 est perdue entre deux pages : mesuré). Décalage 100 000 : 99,7 → 0,168 ms.	Aucun gain à petite profondeur (pages 1 et 2 : ≈ 0,17 ms dans les deux cas). Le curseur ne permet que « suivant » (pas d'accès direct à la page 37), il doit être signé et lié au client, et ce n'est pas une autorisation : le serveur vérifie toujours l'accès (client_id injecté : 400). Mesure sur une table de travail, pas sur ShopFlow.

À retenir. Quand le décalage est grand, le curseur change l'ordre de grandeur ; quand il est petit, OFFSET reste correct et plus simple.

10. Fiche produit : de la lecture SQL au cache Redis

Même fiche, 0 lecture SQL : 2,7 → 0,988 ms (×2,7), avec une copie qui peut dater de 60 s

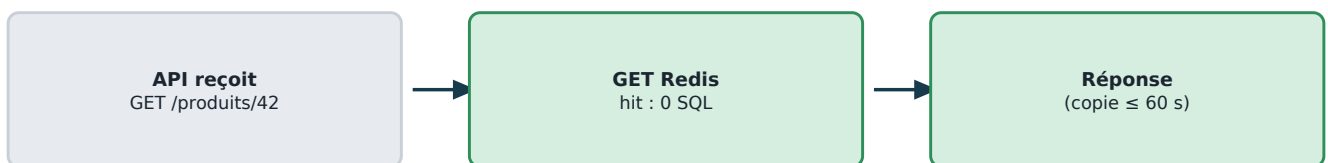
Le problème. La fiche du produit 42 est lue très souvent et change rarement. À chaque lecture l'API envoie un **SELECT** à PostgreSQL pour retrouver exactement la même ligne.

ANCIENNE VERSION	NOUVELLE VERSION
<pre>GET /produits/42 -> SELECT id, nom, prix, stock, attributs FROM produits WHERE id = 42; -- 1 SELECT à chaque lecture</pre>	<pre>GET /produits/42 -> GET Redis shopflow:produit:v1:42 hit : réponse, 0 SQL miss : SELECT, puis SET ... EX 60 PATCH /produits/42 -> UPDATE ... puis DEL de la clé</pre>

ANCIENNE VERSION : ce que fait PostgreSQL



NOUVELLE VERSION : ce que fait PostgreSQL



Requêtes SQL par lecture

Ancienne  1

Nouvelle  0 (hit)

Durée de la requête (ms, 100 paires)

Ancienne  2,7 ms

Nouvelle  0,988 ms (×2,7)

Résultat identique ✓ Le contenu (id, nom, prix, stock, attributs) est **identique** entre le chemin miss et le chemin hit à chacune des 100 paires, et pour les 5 paires de la fiche ; le prix n'a pas bougé pendant la mesure.
Attention : « même résultat » ne vaut que tant que la donnée n'a pas changé (voir le coût).

Pourquoi ça marche	Ce que ça coûte, et les limites
Un hit lit Redis au lieu de PostgreSQL : 0 SQL (un miss en fait 1). Le temps dans l'API passe de 2,1 à 0,671 ms (×3,1). L'écriture par l'API supprime la clé après l'UPDATE, donc la lecture suivante recharge la nouvelle valeur.	La copie peut être périmée : après un UPDATE direct dans PostgreSQL l'API a servi 57.50 alors que la base avait 19.90 (hit, 0 SQL). Si l'invalidation échoue (Redis arrêté pendant un PATCH), le prix est enregistré et la copie ancienne reste. La course de la slide 30 est reproductible. Le gain de durée est petit en local (≈ 1,7 ms) : le vrai gain est la charge évitée sur PostgreSQL . Aucun cache pour l'achat : le prix et le stock sont contrôlés dans la source.

À retenir. Un cache est une optimisation qui peut changer ce que voit l'utilisateur : il faut chiffrer le gain, déclarer la fraîcheur acceptée, et garder le contrôle dans la source.

Transformations écartées, et ce que coûtent les index

Une transformation qui change le résultat ou n'apporte rien est écartée : il faut dire pourquoi

Transformation essayée	Pourquoi écartée
Retirer DISTINCT de la requête sur les clients	Résultat différent : 80 000 lignes au lieu de 800 (le client 42 apparaît 100 fois).
Sommer commandes.total après une jointure avec les lignes de commande	Résultat faux : 143 280 € au lieu de 47 760 €. Plus rapide (1,10 ms contre 2,05 ms) mais incorrect.
Ajouter un index simple sur client_id	Aucun gain : la base avait déjà un index qui commence par client_id ; même plan, même nombre de pages (96).
Indexer tous les statuts pour la file en attente	Fonctionne, mais l'index est 12,4 fois plus gros et WAL +36,5 % : disproportionné pour un besoin limité à un statut.
Ajouter un GIN sur les 200 produits réels	Jamais utilisé (idx_scan = 0) : lire 3 pages coûte moins que passer par l'index.
Laisser le TTL actualiser le prix après un UPDATE direct en SQL	Résultat différent : l'API sert 57.50 (hit, 0 SQL) alors que PostgreSQL a 19.90 ; le TTL (60 s) ne fait que borner la durée de la copie périmée.
Mettre PgBouncer devant l'API pour « aller plus vite »	Pas plus rapide : 40 → 5 connexions PostgreSQL, mais débit 261 → 94 transactions/s (5 serveurs pour 40 clients qui occupent 50 ms). Il limite les connexions, il n'accélère pas.
Écrire l'ORDER BY de l'API sans préfixe de table	L'ORDER BY trie sur des textes : l'index composé devient inutile (0,783 ms, 93 pages, comme sans index).

Le prix d'un index : journal d'écriture (WAL) en plus pour 20 000 insertions

Index partiel (file en attente)	+3,2 %
Index composé (historique client)	+35,0 %
Index d'expression (JSON)	+34,6 %
Index complet (statut, created_at, id)	+36,5 %
Colonne typée + index (JSON)	+38,9 %
GiST (périodes)	+54,1 %
GIN (JSON)	+120,7 %

Le volume de WAL est la mesure fiable du coût d'écriture : il est identique d'une exécution à l'autre, contrairement aux durées. Chaque index ajouté ralentit **toutes** les insertions : on garde le moins d'index possible (import fréquent : limiter les index).

Comment je prouve qu'une optimisation est bonne

Quatre règles appliquées à chaque atelier

1	Même résultat Je compare le résultat complet avant et après (empreinte md5, ou différence symétrique entre deux requêtes). Si le résultat change, la variante est rejetée, même si elle est plus rapide.
2	Mesure répétée 3 échauffements puis 5 mesures (médiane), et des mesures alternées sur 51 tours : les durées de moins d'une milliseconde varient d'une exécution à l'autre.
3	Valeurs stables Je m'appuie sur ce qui ne varie pas : pages lues (hit + read), volume de WAL, taille des index, nœuds du plan. Un gain doit se voir dans le plan, pas seulement dans le chrono.
4	Coût d'écriture Pour chaque index, je mesure ce qu'il ajoute à l'insertion. Une optimisation de lecture qui ruine l'écriture n'est pas retenue sans le dire.
✓	Laboratoire restauré Après chaque expérience, la base retrouve exactement son état initial (6 index, aucune statistique), vérifié par script. Les migrations sont testées sur une base jetable, puis non appliquées.

Limites. Données synthétiques (les volumes de 200 000 lignes sont générés). Durées variables d'une exécution à l'autre. L'API de ShopFlow n'émet pas les requêtes du catalogue ni de la file en attente : j'ai utilisé celles du cours. Deux défauts de mesure ont été trouvés et corrigés en cours de route (buffers incomplets, plans parallèles non comparables).

Pour le détail. Les protocoles, les plans complets et toutes les mesures brutes, atelier par atelier, sont dans Atelier1_Diagnostic_compact.pdf (59 pages). Scripts, résultats et migrations : dossiers atelier3/, atelier4/ et atelier4bis/.